

## IOAI Field

- **Time limit:** 5 minutes
- **Storage:** 5 GB
- **Solution size:** `solution.ipynb`, `custom_model.py`  $\leq 1$  MB together
- **Pretrained models:** none — train from scratch, no internet at grade time
- **Baseline Score:** 31.2187

### Task

The Mayor of Astana wants to decorate the city with stylized IOAI logos. As a statistician, he views everything—including the logo—as a spatial function  $F(x, y, \overline{W})$ , where  $x, y \in [0, 1]$  represent coordinates on a 2D plane and  $\overline{W}$  is a set of hidden parameters defining stylistic attributes such as letter colors and angles.

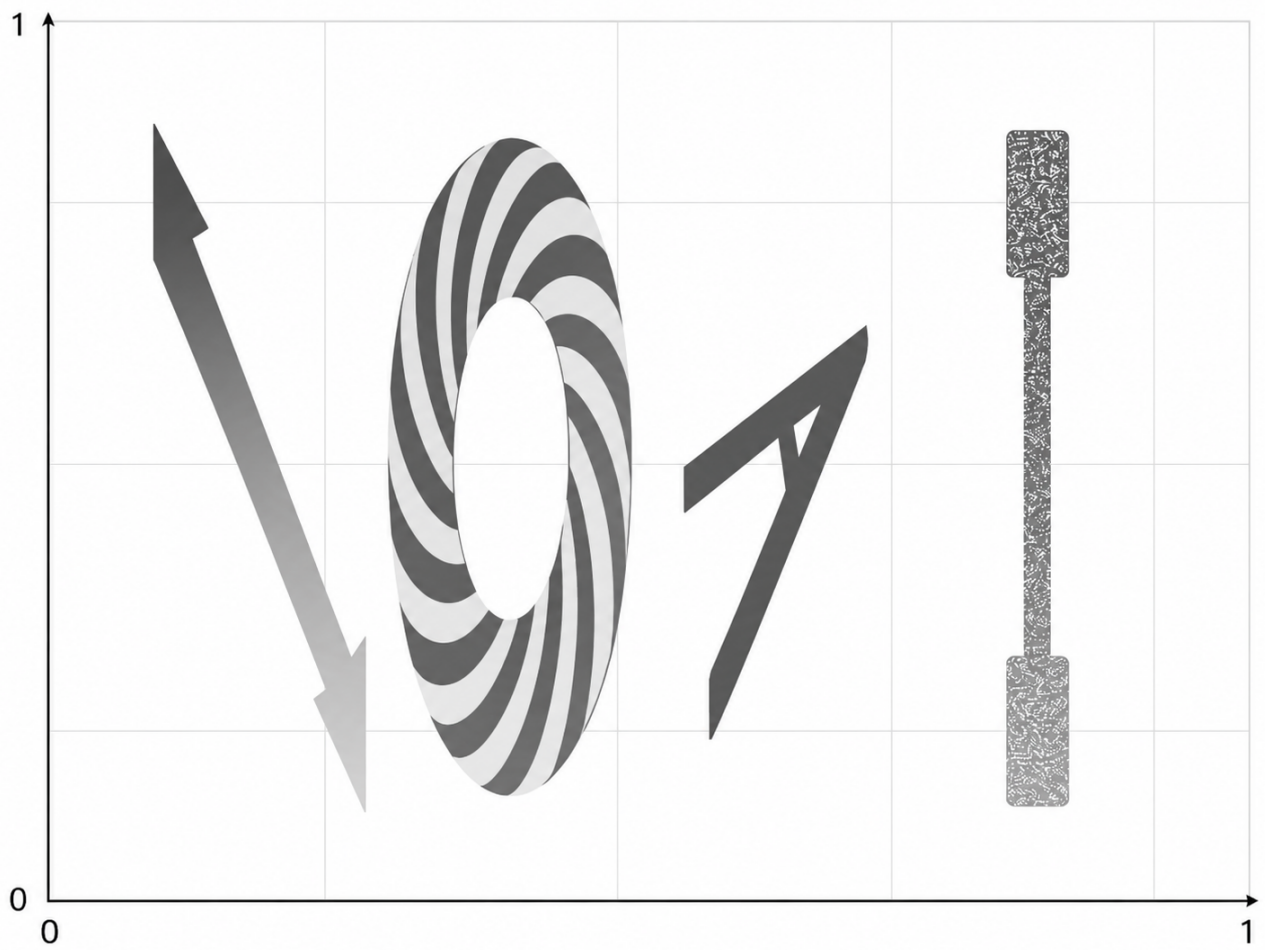
Because  $F$  is too complex to express as an explicit mathematical equation, your task is to train a neural network to approximate it. The network will output an **IOAI field** value for any coordinate pair  $(x, y)$ , generating a complete heatmap visualization of the logo across the plane. Here is an example of heatmap visualization of  $F$  with some specific hidden parameters  $\overline{W}$ .

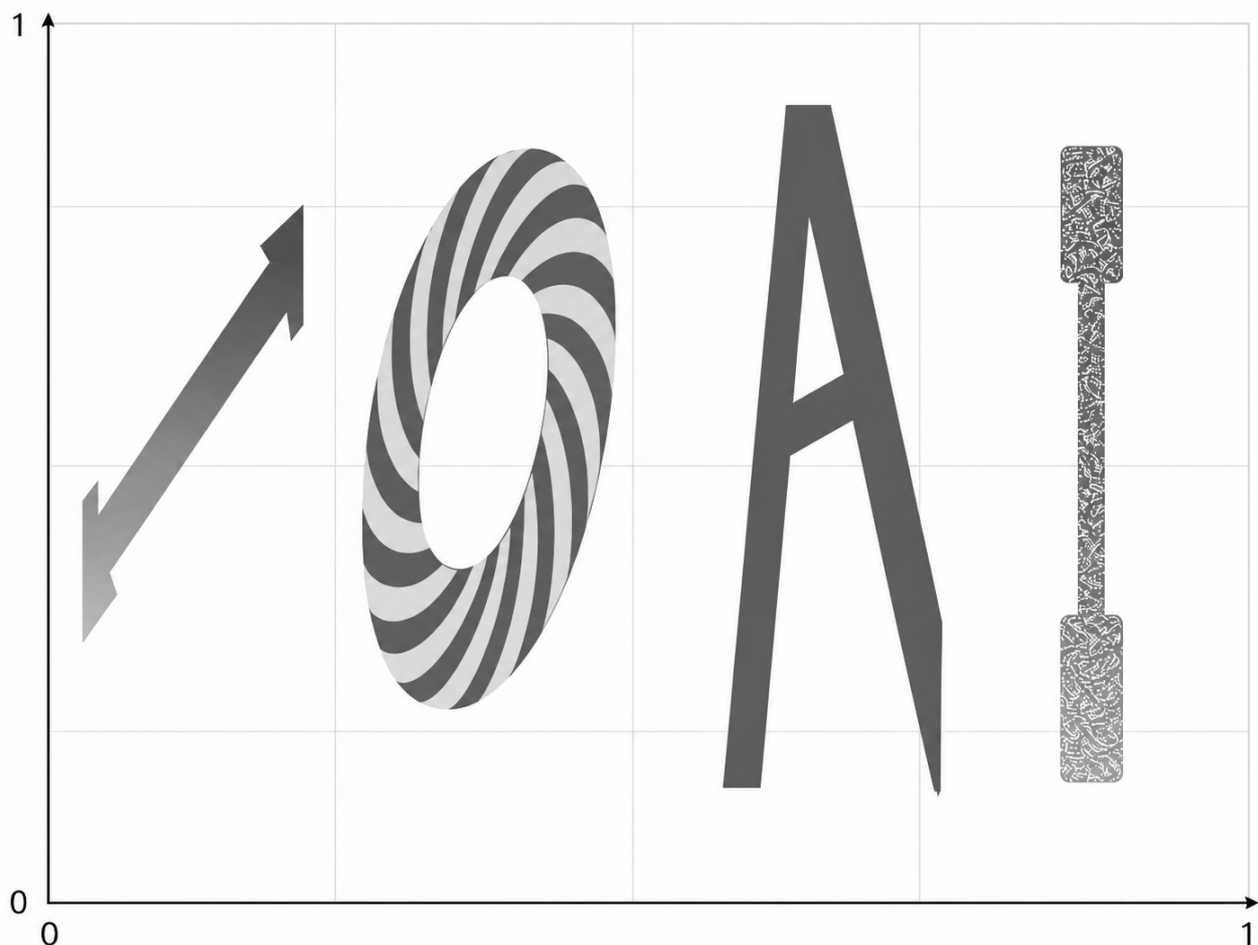


What IOAI field consists of? Four letters and the background.

- Values within the first  $\mathbb{I}$  letter are very large ( $1e+10$  and more) with a linear gradient
- Values in the letter  $\circ$  demonstrate spiral pattern
- Value within the letter  $A$  is always  $-1$
- Values within the last  $\mathbb{I}$  letter should be random values from the range  $[-2026, 2026]$  even if evaluated at the same point twice
- Outside of the letters value is always zero

Function has hidden parameters  $\overline{W}$ , which affects letters scale and incline, together with the range of values within the first  $\mathbb{I}$  letter. However, letters will not intersect. Here are few illustrative examples of how IOAI field looks like with different  $\overline{W}$ :





### What you are given:

This problem contains NO datasets. Instead, you are given the generator function that is configured by the JSON config file at `data/train_config/field_config.json`.

Test config is hidden, but it is of a similar nature. Your task is to fit on the given generator using as many data as you wish. Your "train" and "test" distributions are generated from the same generator - you just don't know on which points  $(x_i, y_i)$  you'll be evaluated.

Your submission should consist of:

- training model class saved as `custom_model.py`. This model should inherit from the `torch.nn.Module` class and use only `torch` imports. It should contain `CustomModel` class used in `solution.ipynb` notebook.
- `solution.ipynb` notebook, which will produce `model.pt` weights

## Scoring

For each region, minimal score is 0 and maximal score is 1. Final score is averaged over all five regions (four for each letter and the background) and multiplied by 100. There is a **parameter penalty**:

**If your model has more than 20260 parameters, the score is halved.**

Number of parameter is measured by `sum(p.numel() for p in model.parameters())`. We expect your model to operate in a stochastic mode as well with the PyTorch `nn.Dropout` being part of the model.

### For Standard Regions

For each region  $R$  (first I letter, O, A, Background), we evaluate the model on  $N_R = 512$  test points  $(x_i, y_i)$  with true values  $v_i$  and predictions  $\hat{v}_i$ . We use normalized Mean Absolute Error (MAE) as a main metric. MAE is defined as:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

And normalization is performed as

$$\text{score}(R) = 1 - \min \left( \frac{\text{MAE}(R)}{s_R}, 1 \right),$$

where  $s_R > 0$  is a scale constant.

### For the last I letter region

In this region, **dropout is enabled during evaluation**. For each test point  $j$ :

1. We run the model  $K = 10$  times to obtain  $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$ .
2. If any output is outside the range  $[-2026, 2026]$ , then  $\text{pointScore}(j) = 0$ .
3. Otherwise, compute the standard deviation  $\sigma_j$  of the  $K$  outputs and convert it to a score:

$$\text{pointScore}(j) = \min \left( \frac{\sigma_j}{s_E}, 1 \right),$$

where  $s_E > 0$  is a fixed scale constant.

The region score is the average over all points in the region:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

where  $N_E = K * N_R$ .

In simple terms, the more diversity you have, the larger your score for this region would be. **You can't use random in pure form, including PyTorch `rand*` and `_uniform` functions, randomness should come from the inference with enabled dropout.**

## How to submit

1. Open `solution.ipynb` and run all cells.
2. Improve the `CustomModel` model in `custom_model.py`
3. Make sure that your last cell saves your model to `model.pt` file.
4. In the JupyterLab Git tab, stage, comment and commit `solution.ipynb` and `custom_model.py`, then push it.
5. Return to the Contest page and click **Submit**. Submit comment should be the same as the comment from the previous step.